

Q1. Average Sale Price using NumPy

Aim

To find the average sale price of houses having more than 4 bedrooms using NumPy.

Algorithm

1. Import NumPy.
2. Create a NumPy array with house details.
3. Filter houses having bedrooms greater than 4.
4. Calculate the average sale price.
5. Display the result.

Code:

```
import numpy as np

house_data = np.array([
    [3,1400,7500000],
    [5,2200,12500000],
    [4,1800,9800000],
    [6,3000,18500000],
    [2,1100,5200000]
])

filtered = house_data[house_data[:,0] > 4]

average_price = np.mean(filtered[:,2])

print("Average Sale Price:", average_price)
```

Output

Average Sale Price: 15500000.0

The screenshot shows two windows from a Windows desktop. The left window is the Python IDLE Shell, titled 'IDLE Shell 3.10.0'. It displays the Python 3.10.0 startup message and the output of a script: 'Average Sale Price: 15500000.0'. The right window is a text editor titled '*q1.py - C:\Users\ASUS\Downloads\fods outputs\q1.py (3.10.0)*'. It contains the following Python code:

```
import numpy as np
house_data = np.array([
    [3, 1400, 7500000],
    [5, 2200, 12500000],
    [4, 1800, 9800000],
    [6, 3000, 18500000],
    [2, 1100, 5200000]
])

filtered = house_data[house_data[:, 0] > 4]
average_price = np.mean(filtered[:, 2])
print("Average Sale Price:", average_price)
```

The taskbar at the bottom shows the Windows Start button, a search bar, and various application icons. The system clock in the bottom right corner indicates the time is 10:38 on 09-07-2022.

Result

The average sale price of houses with more than 4 bedrooms was calculated successfully.

Q2. Spam Email Detection using Logistic Regression

Aim

To classify an email as Spam or Not Spam using Logistic Regression.

Algorithm

1. Import pandas and Logistic Regression.
2. Create the dataset.
3. Train the model.
4. Predict the new email.
5. Display Spam or Not Spam.

Code

```
import pandas as pd
```

```
from sklearn.linear_model import LogisticRegression
```

```

data = {
    'Num_Links':[1,6,0,4,2],
    'Num_Caps_Words':[3,15,1,10,5],
    'Email_Length':[120,300,80,250,160],
    'Spam':[0,1,0,1,0]
}

df = pd.DataFrame(data)

X = df[['Num_Links','Num_Caps_Words','Email_Length']]
y = df['Spam']

model = LogisticRegression()
model.fit(X,y)

new_email = pd.DataFrame({
    'Num_Links':[3],
    'Num_Caps_Words':[8],
    'Email_Length':[200]
})

prediction = model.predict(new_email)

if prediction[0]==1:
    print("Spam Email")
else:
    print("Not Spam")

```

Output

The screenshot shows a Python IDE with two windows. The left window displays a Python script for training and predicting spam emails using Logistic Regression. The right window shows the output of the script, including a warning about feature names and the prediction result.

```
q2.py - C:\Users\ASUS\Downloads\fods outputs\q2.py (3.10.0)
File Edit Format Run Options Window Help
import pandas as pd
from sklearn.linear_model import LogisticRegression

# Dataset
data = {
    'Num_Links': [1, 6, 0, 4, 2],
    'Num_Caps_Words': [3, 15, 1, 10, 5],
    'Email_Length': [120, 300, 80, 250, 160],
    'Spam': [0, 1, 0, 1, 0]
}

df = pd.DataFrame(data)
x = df[['Num_Links', 'Num_Caps_Words', 'Email_Length']]
y = df['Spam']

# Train model
model = LogisticRegression()
model.fit(x, y)

new_email = pd.DataFrame({
    'Num_Links': [3],
    'Num_Caps_Words': [8],
    'Email_Length': [200]
})

prediction = model.predict(new_email)

if prediction[0] == 1:
    print("Spam Email")
else:
    print("Not Spam")

IDLE Shell 3.10.0
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)]
on Win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\ASUS\Downloads\fods outputs\q1.py =====
Average Sale Price: 15500000.0
>>>
===== RESTART: C:\Users\ASUS\Downloads\fods outputs\q2.py =====
Warning (from warnings module):
  File "C:\Users\ASUS\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\
utils\validation.py", line 2749:
    warnings.warn(
UserWarning: X does not have valid feature names, but LogisticRegression was fitted with
feature names
Not Spam
>>>
===== RESTART: C:\Users\ASUS\Downloads\fods outputs\q2.py =====
Not Spam
>>>
```

Result

The Logistic Regression model successfully classified the email.

Q3. Spam Prediction using Logistic Regression

Aim

To predict whether a new email is Spam or Not Spam.

Algorithm

1. Import required libraries.
2. Create the dataset.
3. Train Logistic Regression.
4. Predict the new email.
5. Display prediction.

Code

```
import pandas as pd
```

```
from sklearn.linear_model import LogisticRegression
```

```
data = {
```

```
    'Num_Links':[1,6,0,4,2],
```

```
    'Num_Caps_Words':[3,15,1,10,5],
```

```

    'Email_Length':[120,300,80,250,160],

    'Spam':[0,1,0,1,0]

}

df = pd.DataFrame(data)

X = df[['Num_Links','Num_Caps_Words','Email_Length']]

y = df['Spam']

model = LogisticRegression()

model.fit(X,y)

new_email = pd.DataFrame({

    'Num_Links':[5],

    'Num_Caps_Words':[12],

    'Email_Length':[280]

})

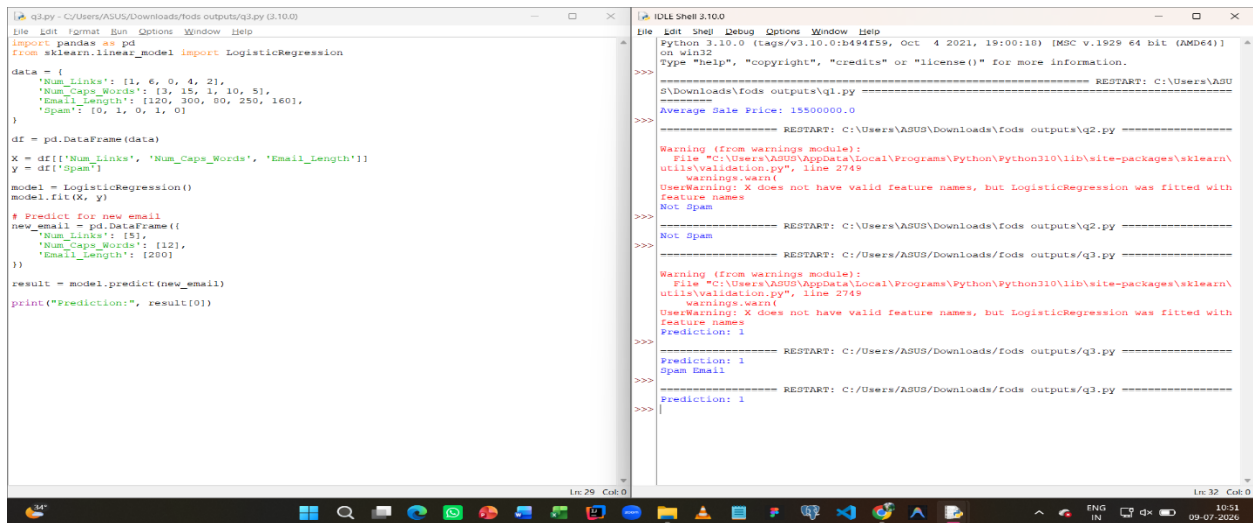
result = model.predict(new_email)

print("Prediction:",result[0])

```

Output

Prediction: 1



```

In [29]: C:\Users\ASUS\Downloads\foods outputs\q3.py (3.10.0)
File Edit Format Run Options Window Help
import pandas as pd
from sklearn.linear_model import LogisticRegression

data = {
    'Num_Links': [1, 6, 0, 4, 2],
    'Num_Caps_Words': [3, 15, 1, 10, 5],
    'Email_Length': [120, 300, 80, 250, 160],
    'Spam': [0, 1, 0, 1, 0]
}

df = pd.DataFrame(data)

X = df[['Num_Links', 'Num_Caps_Words', 'Email_Length']]
y = df['Spam']

model = LogisticRegression()
model.fit(X, y)

# Predict for new email
new_email = pd.DataFrame({
    'Num_Links': [5],
    'Num_Caps_Words': [12],
    'Email_Length': [280]
})

result = model.predict(new_email)
print("Prediction:", result[0])

Out[29]:
Prediction: 1

```

```

Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\ASUS\Downloads\foods outputs\q1.py =====
Average Sale Price: 15500000.0
>>>
===== RESTART: C:\Users\ASUS\Downloads\foods outputs\q2.py =====
Warning (from warnings module):
  File "C:\Users\ASUS\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\utils\validation.py", line 2749
    warnings.warn(
UserWarning: X does not have valid feature names, but LogisticRegression was fitted with feature names
Not Spam
>>>
===== RESTART: C:\Users\ASUS\Downloads\foods outputs\q2.py =====
Warning (from warnings module):
  File "C:\Users\ASUS\AppData\Local\Programs\Python\Python310\lib\site-packages\sklearn\utils\validation.py", line 2749
    warnings.warn(
UserWarning: X does not have valid feature names, but LogisticRegression was fitted with feature names
Prediction: 1
>>>
===== RESTART: C:\Users\ASUS\Downloads\foods outputs\q3.py =====
Prediction: 1
Spam Email
>>>
===== RESTART: C:\Users\ASUS\Downloads\foods outputs\q3.py =====
Prediction: 1
>>>

```

Result

The Logistic Regression model predicted the new email as Spam (1).

Q4. Car Resale Price Prediction using Linear Regression

Aim

To predict the resale price of a car using Linear Regression.

Algorithm

1. Import pandas and Linear Regression.
2. Create the dataset.
3. Train the model.
4. Get user input.
5. Predict resale price.
6. Display the estimated price.

Code

```
import pandas as pd

from sklearn.linear_model import LinearRegression

data={

'Age':[2,5,3,7,1],

'Kms_Driven':[25000,60000,40000,90000,15000],

'Engine_CC':[1200,1500,1300,1800,1000],

'Resale_Price':[650000,420000,520000,300000,720000]

}

df=pd.DataFrame(data)

X=df[['Age','Kms_Driven','Engine_CC']]

y=df['Resale_Price']
```

```

model=LinearRegression()

model.fit(X,y)

age=int(input("Enter Car Age (Years): "))

kms=int(input("Enter Kms Driven: "))

engine=int(input("Enter Engine CC: "))

new_car=pd.DataFrame({

'Age':[age],

'Kms_Driven':[kms],

'Engine_CC':[engine]

})

prediction=model.predict(new_car)

print("Estimated Resale Price = ₹{:.2f}".format(prediction[0]))

```

Sample Input

Enter Car Age (Years): 2

Enter Kms Driven: 12000

Enter Engine CC: 1200

Output

Estimated Resale Price = ₹644105.26

```

q4.py - C:/Users/ASUS/Downloads/tods outputs/q4.py (3.10.0)
File Edit Format Run Options Window Help
import pandas as pd
from sklearn.linear_model import LinearRegression
data = [
    {'Age': [2, 3, 3, 7, 1],
     'Kms_Driven': [25000, 60000, 40000, 90000, 15000],
     'Engine_CC': [1200, 1500, 1300, 1000, 1000],
     'Resale_Price': [650000, 420000, 520000, 300000, 720000]}]
df = pd.DataFrame(data)
X = df[['Age', 'Kms_Driven', 'Engine_CC']]
y = df['Resale_Price']
model = LinearRegression()
model.fit(X, y)
age = int(input("Enter Car Age (Years): "))
kms = int(input("Enter Kms Driven: "))
engine = int(input("Enter Engine CC: "))
new_car = pd.DataFrame({
    'Age': [age],
    'Kms_Driven': [kms],
    'Engine_CC': [engine]
})
prediction = model.predict(new_car)
print("Estimated Resale Price = ₹{:.2f}".format(prediction[0]))

```

```

Warning (from warnings module):
  File "C:/Users/ASUS/AppData/Local/Programs/Python/Python310/lib/site-packages/sklearn/
utils/validation.py", line 2749
    warnings.warn(
UserWarning: X does not have valid feature names, but LogisticRegression was fitted with
feature names
Prediction: 3
>>> RESTART: C:/Users/ASUS/Downloads/tods outputs/q3.py -----
Prediction: 1
Open Shell
>>> RESTART: C:/Users/ASUS/Downloads/tods outputs/q3.py -----
Prediction: 1
>>> RESTART: C:/Users/ASUS/Downloads/tods outputs/q4.py -----
Enter Car Age: 2
Enter Kms Driven: 25000
Enter Engine CC: 1200
Warning (from warnings module):
  File "C:/Users/ASUS/AppData/Local/Programs/Python/Python310/lib/site-packages/sklearn/
utils/validation.py", line 2749
    warnings.warn(
UserWarning: X does not have valid feature names, but LinearRegression was fitted with f
eature names
Estimated Resale Price = ₹ 633157.8947368423
>>> RESTART: C:/Users/ASUS/Downloads/tods outputs/q4.py -----
Enter Car Age:
Enter Car Age: 2
Enter Kms Driven: 15
Enter Engine CC: 2600
Traceback (most recent call last):
  File "C:/Users/ASUS/Downloads/tods outputs/q4.py", line 5, in <module>
    new_car = pd.DataFrame(
NameError: name 'pd' is not defined. Did you mean: 'id'?
>>> RESTART: C:/Users/ASUS/Downloads/tods outputs/q4.py -----
Enter Car Age (Years): 2
Enter Kms Driven: 12000
Enter Engine CC: 1200
Estimated Resale Price = ₹644105.26

```

Result

The Linear Regression model successfully predicted the resale price of the car.

Q5. Loan Approval using CART Decision Tree

Aim

To predict whether a loan application is Approved or Rejected using CART Decision Tree.

Algorithm

1. Import pandas and Decision Tree Classifier.
2. Create the dataset.
3. Train the CART model.
4. Accept applicant details.
5. Predict approval.
6. Display decision rules.

Code

```
import pandas as pd

from sklearn.tree import DecisionTreeClassifier, export_text

data={

'Income':[45000,30000,60000,28000,52000],

'Credit_Score':[720,650,780,620,710],

'Existing_Loans':[0,1,0,2,1],

'Approved':[1,0,1,0,1]

}

df=pd.DataFrame(data)

X=df[['Income','Credit_Score','Existing_Loans']]

y=df['Approved']
```



```

model=DecisionTreeClassifier(random_state=42)
model.fit(X,y)
income=int(input("Enter Income: "))
credit=int(input("Enter Credit Score: "))
loans=int(input("Enter Existing Loans: "))
new_applicant=pd.DataFrame({
'Income':[income],
'Credit_Score':[credit],
'Existing_Loans':[loans]
})
prediction=model.predict(new_applicant)
if prediction[0]==1:
    print("Loan Approved")
else:
    print("Loan Rejected")
print(export_text(model,
feature_names=['Income','Credit_Score','Existing_Loans']))

```

Sample Input

Enter Income: 50000

Enter Credit Score: 741

Enter Existing Loans: 0

Output

Loan Approved

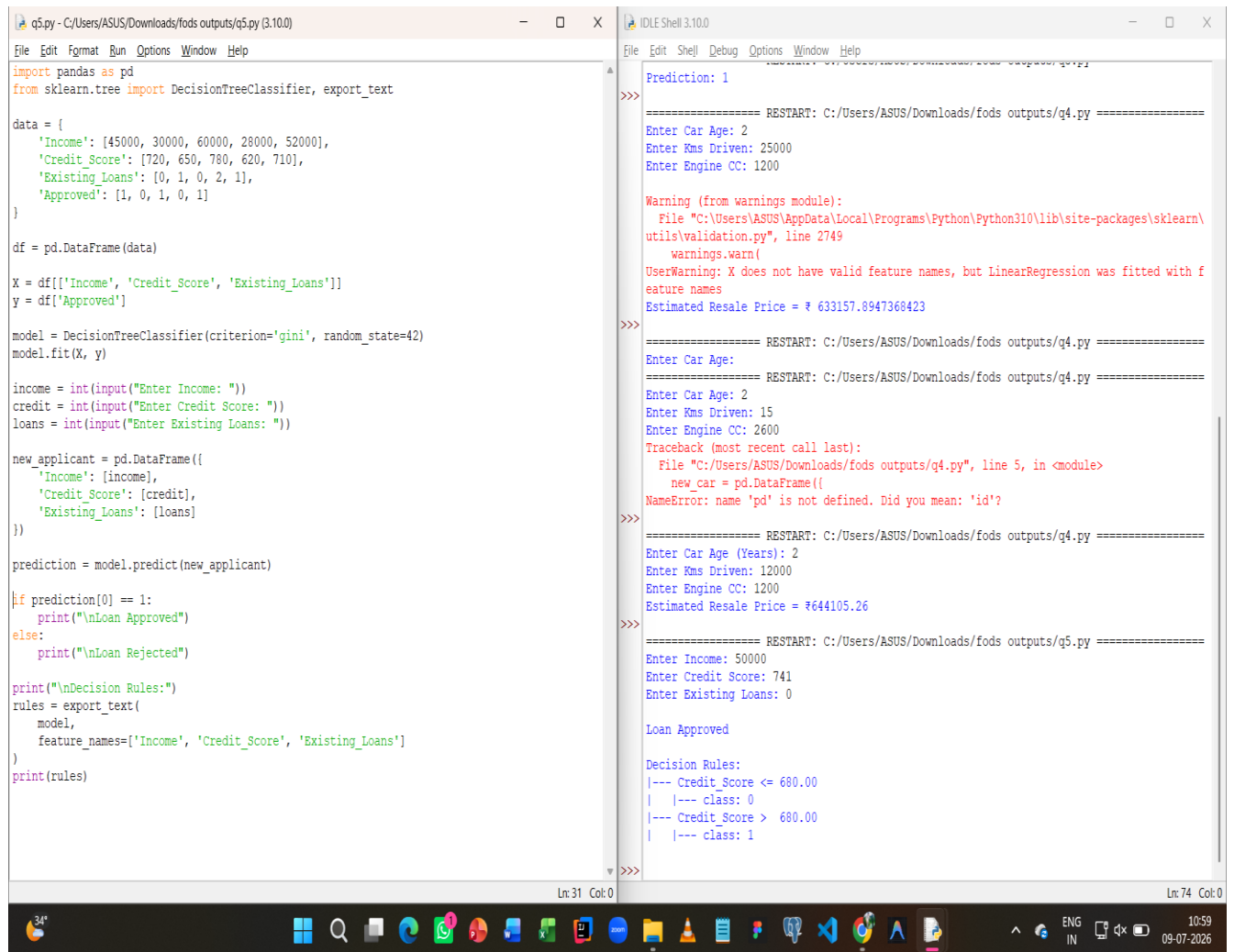
Decision Rules:

|--- Credit_Score <= 680.00

| |--- class: 0

|--- Credit_Score > 680.00

| |--- class: 1



The screenshot shows a Python script in a text editor and its execution in the IDLE Shell. The script defines a dataset with features: Income, Credit_Score, Existing_Loans, and Approved. It uses a DecisionTreeClassifier to model the data. The script then prompts the user for input (Income, Credit Score, Existing Loans) and predicts the loan approval status. Finally, it exports the decision rules.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text

data = {
    'Income': [45000, 30000, 60000, 28000, 52000],
    'Credit_Score': [720, 650, 780, 620, 710],
    'Existing_Loans': [0, 1, 0, 2, 1],
    'Approved': [1, 0, 1, 0, 1]
}

df = pd.DataFrame(data)

X = df[['Income', 'Credit_Score', 'Existing_Loans']]
y = df['Approved']

model = DecisionTreeClassifier(criterion='gini', random_state=42)
model.fit(X, y)

income = int(input("Enter Income: "))
credit = int(input("Enter Credit Score: "))
loans = int(input("Enter Existing Loans: "))

new_applicant = pd.DataFrame({
    'Income': [income],
    'Credit_Score': [credit],
    'Existing_Loans': [loans]
})

prediction = model.predict(new_applicant)

if prediction[0] == 1:
    print("\nLoan Approved")
else:
    print("\nLoan Rejected")

print("\nDecision Rules:")
rules = export_text(
    model,
    feature_names=['Income', 'Credit_Score', 'Existing_Loans']
)
print(rules)
```

The IDLE Shell output shows the execution of the script. It displays the prediction for a new applicant (Loan Approved) and the decision rules. The rules are:

```
|--- Credit_Score <= 680.00
| |--- class: 0
| |--- Credit_Score > 680.00
| |--- class: 1
```

Result

The CART Decision Tree model successfully predicted the loan approval status and displayed the decision rules